UCMIS M2M Transformation — Project Documentation

Version: 1.0
Last Updated: 2026-03-05
Compatibility: Eclipse 2025-12 · QVTo 3.11.1 · Java 21

Table of Contents

1. Summary for a Non-Technical Audience
2. Requirements
3. Usage: Building and Running the Tool
4. Where to Make Changes
5. Complete Technical Reference
    - 5.1 Project Architecture Overview
    - 5.2 Module: `ucmism2m.blackbox`
    - 5.3 Module: `ucmism2m.transformation`
    - 5.4 Module: `ucmism2m.app`
    - 5.5 Module: `ucmism2m.product`
    - 5.6 Module: `ucmism2m.target`
    - 5.7 The Mapping Configuration JSON
    - 5.8 Transformation Execution Flow
    - 5.9 Output Model Structure
    - 5.10 Provenance and SSSOM
    - 5.11 Build System
    - 5.12 QVTo Constraints and Design Rules

---

1. Summary for a Non-Technical Audience

What does this tool do?

The UCMIS M2M Transformation tool converts a UML model (a formal diagram describing the structure of data and concepts) from one representation into another. Specifically, it reads the **DDI-CDI** model — a widely used international standard for describing survey and research data concepts — and produces a **profile model** (such as DDSC or CDIF) that contains a carefully chosen subset of the DDI-CDI concepts, adapted to the requirements of that profile.

This conversion would traditionally require a data architect to manually replicate dozens of class definitions, attributes, and relationships across two large model files. The tool automates this fully: a single configuration file describes which concepts to include and how they should be mapped, and the tool does the rest.

Why is this useful?

Different communities working with research data (statisticians, social scientists, archivists) use different but related conceptual models. The DDI-CDI model is the common reference point. Profile models derived from it must stay consistent with DDI-CDI while being lighter and more focused. Maintaining this consistency by hand is error-prone and time-consuming. This tool makes it repeatable, transparent, and auditable.

What does the output look like?

The output is a standard UML model file (`.uml` format) that can be opened in any UML modelling tool that supports Eclipse UML2, such as Papyrus or Eclipse itself. The output contains:

- All selected classes, with their attributes and definitions.
- All selected associations between classes.
- A metadata block recording the model's name, version, language, and URI.
- Provenance records — machine-readable links back to the DDI-CDI concepts each profile concept was derived from, in both human-readable and RDF Turtle formats.

What does a user need to do?

The only file a user normally needs to edit is the **JSON configuration file** (described in detail in section 5.7). This file lists which classes, attributes, and associations from DDI-CDI should appear in the profile model, and allows overriding names, definitions, and multiplicities as needed. The tool is then run from the command line. No programming is required to operate the tool once it is built.

---

2. Requirements

2.1 Runtime Requirements

The built product is a self-contained, headless Eclipse application. It includes all necessary Eclipse and EMF runtime libraries. The only external requirement to run the built binary is:

Requirement	Version	Notes
Java (JRE or JDK)	21 or later	OpenJDK or Oracle JDK both supported

No Eclipse installation is needed at runtime. The tool is distributed as a directory containing a launcher executable and a `plugins/` directory.

2.2 Build Requirements

To build the tool from source, the following are required:

Requirement	Version	Notes
Java Development Kit (JDK)	21	Must be JDK (not just JRE)
Apache Maven	3.9 or later	Standard Maven installation
Internet access	—	Required to download Eclipse p2 dependencies during first build

The build system uses **Eclipse Tycho** (a Maven plugin for building Eclipse applications). Tycho downloads all Eclipse dependencies automatically from the configured p2 repositories. No manual Eclipse installation is required to build.

2.3 Eclipse IDE Requirements (Development Only)

For working on the source code in Eclipse:

Requirement	Version
Eclipse IDE for RCP and RAP Developers	2025-12
Eclipse QVTo SDK (installed via Help → Install New Software)	3.11.1
Eclipse UML2 (installed via Help → Install New Software)	5.x (part of Eclipse Modeling Tools)

The target platform file `ucmism2m.target/ucmism2m.target.target` must be activated in Eclipse before any development work (Window → Preferences → Plug-in Development → Target Platform → activate `UCMIS M2M Target Platform`).

---

3. Usage: Building and Running the Tool

3.1 Building

From the project root directory (the directory containing the top-level `pom.xml`), run:

```bash
# Build for the current platform only (Linux x86_64 by default — fastest)
mvn clean verify

# Build for all platforms (Linux, Windows, macOS x86_64 and ARM)
mvn clean verify -P all-platforms

# Build for Windows only
mvn clean verify -P windows-only

# Build for macOS only
mvn clean verify -P macos-only
```

The build output is placed in:

```
ucmism2m.product/target/products/ucmism2m/
```

Platform-specific ZIP archives are also created in that directory:

Archive	Platform
`ucmism2m-linux.gtk.x86_64.zip`	Linux 64-bit
`ucmism2m-win32.win32.x86_64.zip`	Windows 64-bit
`ucmism2m-macosx.cocoa.x86_64.zip`	macOS Intel
`ucmism2m-macosx.cocoa.aarch64.zip`	macOS Apple Silicon

3.2 Location of Binaries

After building and extracting the ZIP for your platform, the product directory has the following layout:

```
ucmism2m/                  ← root of the extracted archive
├── ucmism2m               ← launcher executable (Linux/macOS)
├── ucmism2m.exe           ← launcher executable (Windows)
├── plugins/               ← all OSGi bundles
│   ├── ucmism2m.app_1.0.0...jar
│   ├── ucmism2m.blackbox_1.0.0...jar
│   ├── ucmism2m.transformation_1.0.0...jar
│   └── ... (Eclipse runtime bundles)
└── configuration/
    └── config.ini
```

The main executable is `ucmism2m` (Linux/macOS) or `ucmism2m.exe` (Windows) in the root of the extracted directory.

3.3 Running the Tool

```bash
./ucmism2m \
  -input  /path/to/source-model.uml \
  -output /path/to/output-model.uml \
  -config /path/to/mapping-config.json
```

Arguments:

Argument	Required	Description
`-input <path>`	Yes	Path to the source UML model file (`.uml` format, Eclipse UML2)
`-output <path>`	Yes	Path where the output UML model will be written (`.uml` format)
`-config <path>`	Yes	Path to the JSON mapping configuration file

Example (DDI-CDI to DDSC):

```bash
./ucmism2m \
  -input  /data/models/DDI-CDI.uml \
  -output /data/models/DDSC.uml \
  -config /data/configs/ddi-cdi_to_ddsc.json
```

3.4 Interpreting Output

The tool prints progress to standard output:

```
UCMIS M2M Transformation Application
Eclipse 2025-12 | QVTo 3.11.1 | Java 21
====================================
Input model: /data/models/DDI-CDI.uml
Output model: /data/models/DDSC.uml
Configuration: /data/configs/ddi-cdi_to_ddsc.json

Loading input model...
Input model loaded: 1 root elements
Loading transformation...
Transformation loaded successfully

Executing transformation...
Transformation completed successfully!

Saving output model...
Output model saved: /data/models/DDSC.uml
```

If the transformation fails, an error and stack trace are printed to standard error and the process exits with code 1.

3.5 Cache Clearing (Development)

The Eclipse OSGi framework caches extracted bundle content. If QVTo script changes appear to have no effect after a rebuild, the cache must be cleared. This is handled automatically during `mvn clean verify` via the maven-clean-plugin configuration in `ucmism2m.product/pom.xml`, which deletes the OSGi cache directory. Manual cache clearing can be done by deleting:

```
ucmism2m.product/target/products/ucmism2m/<platform>/configuration/org.eclipse.osgi/
```

---

4. Where to Make Changes

4.1 Mapping Definitions (QVTo Scripts)

All transformation logic is in the `ucmism2m.transformation` module:

```
ucmism2m.transformation/transforms/
├── m2m.qvto        ← Main transformation script (entry point, orchestration)
├── ConfigLib.qvto   ← Blackbox helper declarations (binds QVTo to Java)
├── helpers.qvto     ← Lookup queries, multiplicity helpers, comment helpers
└── mappings.qvto    ← (legacy; mappings are now consolidated in m2m.qvto)
```

`m2m.qvto` is the primary file to edit for transformation changes. It contains:

- The `main()` function: the top-level orchestration of all 11 transformation phases.
- All mapping operations: `mapClass`, `mergeClass`, `newClass`, `copyAttribute`, `copyOneDataType`, `copySubDataType`, `createAssociation`, `createDependencies`, and their helpers.
- All query helper functions: model lookup, multiplicity conversion, comment handling, DataType hierarchy traversal.

When to edit `m2m.qvto`:

- To change how classes are created or which class attributes are copied.
- To change how associations are structured or their navigability.
- To change how DataTypes (including their subtype hierarchies) are reproduced.
- To add new transformation phases or entirely new kinds of model elements.
- To change how provenance dependencies are recorded.
- To fix or extend comment / definition handling.

`ConfigLib.qvto` must be kept in sync with `JSONConfigLoader.java`. It declares the signatures of all Java blackbox operations so QVTo can call them. Adding a new Java method in `JSONConfigLoader` requires a matching `blackbox helper` declaration in `ConfigLib.qvto`.

`helpers.qvto` contains model lookup queries and multiplicity/comment utilities that are shared across multiple mapping operations. These rarely need changing unless a new category of source or target model element is introduced.

4.2 Blackbox Java Operations

All Java-side configuration reading and proxy creation is in the `ucmism2m.blackbox` module:

```
ucmism2m.blackbox/src/blackbox/
├── JSONConfigLoader.java        ← All blackbox operation implementations
└── config/
    ├── ParsedConfig.java        ← Root POJO for the JSON config
    ├── TransformationConfig.java ← Transformation metadata POJO
    ├── SourceModelConfig.java   ← Source model identity POJO
    ├── TargetModelConfig.java   ← Target model identity + structure POJO
    ├── PackageConfig.java       ← Additional package definition POJO
    ├── ClassMappingConfig.java  ← Class mapping entry POJO
    ├── AttributeConfig.java     ← Attribute selection entry POJO
    ├── AssociationMappingConfig.java ← Association mapping entry POJO
    ├── MultiplicityConfig.java  ← Multiplicity override POJO
    ├── SssomConfig.java         ← SSSOM provenance metadata POJO
    └── SourceClassDeserializer.java ← Jackson custom deserialiser
```

`JSONConfigLoader.java` is the only file to edit for blackbox changes. It is the sole interface between the JSON configuration and the QVTo transformation engine.

When to edit `JSONConfigLoader.java`:

- To expose a new field from the configuration JSON to QVTo (add a new `@Operation` public static method, then add a matching `blackbox helper` in `ConfigLib.qvto`).
- To change how provenance comment bodies are formatted.
- To change how supplier proxy URIs are constructed.
- To change how the configuration is loaded or cached.

When to edit the `config/` POJO classes:

- When the JSON configuration schema gains new fields. Add the field to the appropriate POJO with a `@JsonProperty` annotation and a getter. No changes to `JSONConfigLoader` are needed for simple passthrough fields; only new accessor methods on `JSONConfigLoader` need to be added.

When to edit `UCMISTransformationApp.java` (in `ucmism2m.app`):

- To change how the output file is serialised (e.g. switching from random UUIDs to deterministic hashed `xmi:id` values).
- To change command-line argument parsing.
- To change EMF resource set configuration (e.g. registering additional resource factories).
- To change QVTo execution context setup (e.g. adding logging verbosity).

4.3 Summary Table

Task	File to Edit	Module
Add/change a transformation phase	m2m.qvto	ucmism2m.transformation
Add/change how a model element is created	m2m.qvto	ucmism2m.transformation
Add a new Java accessor method visible to QVTo	JSONConfigLoader.java + ConfigLib.qvto	ucmism2m.blackbox + ucmism2m.transformation
Add a new JSON configuration field	config/<X>Config.java + JSONConfigLoader.java + ConfigLib.qvto	ucmism2m.blackbox + ucmism2m.transformation
Change output XMI serialisation	UCMISTransformationApp.java	ucmism2m.app
Add/change build platform targets	pom.xml (parent)	root
Change Eclipse target platform dependencies	ucmism2m.target.target	ucmism2m.target

---

5. Complete Technical Reference

5.1 Project Architecture Overview

The project is a multi-module Maven/Tycho build producing a headless Eclipse application (Eclipse RCP product without a UI). It is organised as six OSGi bundles:

```
ucmism2m/                    ← Parent Maven project
├── ucmism2m.target/         ← Eclipse target platform definition
├── ucmism2m.blackbox/       ← Java blackbox operations (JSON config reading)
├── ucmism2m.transformation/ ← QVTo transformation scripts
├── ucmism2m.app/            ← Eclipse IApplication entry point
├── ucmism2m.feature/        ← Eclipse feature grouping all three bundles
└── ucmism2m.product/        ← Eclipse product + platform assembly
```

The three functional bundles have the following dependency order:

```
ucmism2m.app
  └── requires ucmism2m.transformation
      └── requires ucmism2m.blackbox
```

Key technology stack:

Layer	Technology	Version
Model metamodel	Eclipse UML2	5.0.0
Transformation language	QVTo (QVT Operational)	3.11.1
JSON parsing	Jackson ObjectMapper	2.20.1
OSGi runtime	Eclipse Equinox	(Eclipse 2025-12)
Build	Maven + Eclipse Tycho	5.0.0
Java	Java 21	—

5.2 Module: `ucmism2m.blackbox`

Bundle symbolic name: `ucmism2m.blackbox`
Purpose: Provides all Java operations that QVTo cannot perform natively — specifically, reading and interpreting the JSON mapping configuration file, generating provenance comment text, and creating EMF proxy objects.

5.2.1 JSONConfigLoader

`src/blackbox/JSONConfigLoader.java`

This is the central class of the blackbox module. It is registered with the QVTo runtime via the `javaBlackboxUnits` extension point in `plugin.xml`. All public methods are annotated `@Operation(contextual=false)`, which means QVTo calls them as free functions (not as methods on a receiver object).

Configuration cache: The class maintains a static `ConcurrentHashMap<String, ParsedConfig>` that caches parsed configurations by canonical file path. The first call for a given path parses the JSON and caches the result; all subsequent calls in the same JVM session are served from cache. This means the JSON file is read exactly once per transformation run, regardless of how many times individual accessor methods are called.

Key methods by category:

Source model metadata:

- `getSourceModelName(configPath)` — Name of the source model (e.g. `"DDI-CDI"`).
- `getSourceModelMainPackage(configPath)` — Main package name in the source model.
- `getSourceModelUri(configPath)` — URI namespace of the source model.

Target model metadata (also become attributes on the `metadata` DataType):

- `getTargetModelAcronym(configPath)` — Short acronym (also used as root Model name).
- `getTargetModelTitle(configPath)` — Full model title.
- `getTargetModelSubTitle(configPath)` — Subtitle.
- `getTargetModelMajorVersion(configPath)`, `getTargetModelMinorVersion(configPath)` — Version strings.
- `getTargetModelLanguage(configPath)` — Language code (e.g. `"en"`).
- `getTargetModelUri(configPath)` — URI namespace.
- `getTargetModelDefinition(configPath)` — Definition text for the root Model comment.
- `getTargetModelMainPackage(configPath)` — Main package name.
- `getTargetModelMainPackageDefinition(configPath)` — Definition text for main package comment.
- `getTargetModelDataTypesPackage(configPath)` — Package where copied DataTypes land.

Package definitions:

- `getDefinedPackageNames(configPath)` → `List<String>` — All extra package names in config order.
- `getDefinedPackageParent(configPath, packageName)` — Parent package name for a given extra package.
- `getDefinedPackageDefinition(configPath, packageName)` — Definition text for a package.

Class mapping — iteration:

- `getMapSourceClasses(configPath)` → `List<String>` — All `sourceClass` values from `"map"` entries.
- `getMergeTargetClasses(configPath)` → `List<String>` — All `targetClass` values from `"merge"` entries.
- `getNewTargetClasses(configPath)` → `List<String>` — All `targetClass` values from `"new"` entries.

Class mapping — resolution:

- `getTargetClassForSourceClass(configPath, sourceClass)` — Target class name for a mapped source class.
- `getTargetPackageForSourceClass(configPath, sourceClass)` — Target package for a `"map"` mapping.
- `getTargetPackageForTargetClass(configPath, targetClass)` — Target package for `"merge"` or `"new"`.
- `getMergeSourceClasses(configPath, targetClass)` → `List<String>` — Both source classes in a merge.
- `getMergeDefinitionFromClass(configPath, targetClass)` — Which source class provides the definition.
- `getNewClassDefinition(configPath, targetClass)` — Definition text for a `"new"` class.

Attribute selection:

- `getClassAttributeNames(configPath, classKey)` → `List<String>` — Ordered attribute names.
- `getMergeAttributeSourceClass(configPath, targetClass, attrName)` — Source class for a merge attribute.
- `getAttributeLower(configPath, classKey, attrName)` — Lower multiplicity override or `""`.
- `getAttributeUpper(configPath, classKey, attrName)` — Upper multiplicity override or `""`.
- `getAttributeDefinition(configPath, classKey, attrName)` — Definition override or `""`.

Association mapping:

- `getTargetAssociationNames(configPath)` → `List<String>` — All target association triple names.
- `getSourceAssociationName(configPath, targetAssocName)` — Corresponding source association name.
- `getAssociationSubjectClass(configPath, targetAssocName)` — Subject class (first triple segment).
- `getAssociationObjectClass(configPath, targetAssocName)` — Object class (last triple segment).
- `getAssociationPredicate(configPath, targetAssocName)` — Predicate (middle segment).
- `getAssociationObjectLower/Upper(configPath, targetAssocName)` — Object end multiplicity overrides.
- `getAssociationSubjectLower/Upper(configPath, targetAssocName)` — Subject end multiplicity overrides.
- `getAssociationDefinition(configPath, targetAssocName)` — Definition override or `""`.

Provenance:

- `getSupplierUri(configPath, sourceClass)` — Constructs `sourceModel.uri + "#" + sourceClass`.
- `buildProvenanceCommentBody(configPath, targetClass, sourceClass)` — Generates RST + RDF Turtle provenance text.
- `createSupplierProxy(supplierUri)` — Creates an unresolved EMF `uml:Class` proxy with the given URI.

5.2.2 Association triple name convention

Association names follow the UCMIS convention: `SubjectClass_predicate_ObjectClass`. The private method `parseTriple()` in `JSONConfigLoader` splits on the first and last underscore, so compound predicates that do not themselves contain underscores are handled correctly. Example:

```
RepresentedVariable_takesSubstantiveValuesFrom_SubstantiveValueDomain
  → subject: "RepresentedVariable"
  → predicate: "takesSubstantiveValuesFrom"
  → object: "SubstantiveValueDomain"
```

5.2.3 Configuration POJO classes

All classes in `src/blackbox/config/` are Jackson-annotated POJOs. They use `@JsonIgnoreProperties(ignoreUnknown=true)` so the configuration can carry `$schema`, `_comment`, or forward-compatible fields without causing parse failures.

`SourceClassDeserializer` handles the JSON Schema `oneOf(String, array)` pattern used for the `sourceClass` field: a plain string for `"map"` mappings and a two-element array for `"merge"` mappings. It normalises both to `List<String>`.

5.2.4 OSGi registration

`plugin.xml` registers `JSONConfigLoader` under the `org.eclipse.m2m.qvt.oml.javaBlackboxUnits` extension point, making it visible to the QVTo runtime without requiring an explicit `import` in the QVTo scripts. The `ucmism2m.transformation` bundle requires `ucmism2m.blackbox` via its `MANIFEST.MF`, ensuring the blackbox is on the OSGi classpath when the transformation runs.

5.3 Module: `ucmism2m.transformation`

Bundle symbolic name: `ucmism2m.transformation`
Purpose: Contains all QVTo transformation scripts. This module is the transformation engine.

5.3.1 File structure

```
transforms/
├── m2m.qvto        ← Transformation entry point + all mappings
├── ConfigLib.qvto  ← Blackbox helper declarations
└── helpers.qvto    ← Lookup queries and utilities
```

QVTo imports use bundle-relative paths: `import ConfigLib;` resolves to `platform:/plugin/ucmism2m.transformation/transforms/ConfigLib.qvto`.

5.3.2 `m2m.qvto` — transformation header

```
qvto

import ConfigLib;
transformation m2m(in inModel : UML, out outModel : UML);
modeltype UML uses 'http://www.eclipse.org/uml2/5.0.0/UML';
configuration property configPath : String;
property copiedTypeNames : OrderedSet(String) = OrderedSet{};
```

- `inModel` is the global input extent. It must not be shadowed by local parameter names.
- `outModel` is the global output extent. Queries that scan target model objects use `outModel.rootObjects()`.
- `configPath` is the only configuration property. It is passed by `UCMISTransformationApp` via `executionContext.setConfigProperty("configPath", configPath)`.
- `copiedTypeNames` is a module-level property (not a local variable) so it persists across all `copyOneDataType` calls in a single run. It tracks DataType names already copied to prevent duplication and infinite recursion.

5.3.3 Transformation phases in `main()`

The `main()` function executes eleven sequential phases:

Phase	Operation	Description
1	`createTargetModel`	Creates the root `uml:Model` element with the target model acronym as name and its definition as a UML comment.
2	`createMainPackage`	Creates the main package (e.g. `DDSC`) directly under the root model.
3	`createMetadataPackage`	Creates the `metadata` package containing the `metadata` DataType with seven read-only String attributes recording model identity.
4	Additional packages	Iterates `getDefinedPackageNames(configPath)` and creates each extra package under its declared parent (inlined in `main()` to access `rootModel` as a local variable).
5	`mapClass`	Creates one target class per `"map"` source class, copying the class definition and selected attributes.
6	`mergeClass`	Creates one target class per `"merge"` entry, drawing attributes from two source classes.
7	`newClass`	Creates target classes with no source equivalent.
8–9	`copyDataTypes`	Post-pass: scans all target class attributes, copies needed DataTypes from the source model, and updates type references to the target copies.
10	`createAssociation`	Creates one UML Association per entry in the config's association array.
11	`createDependencies`	Creates provenance Dependency elements for all `"map"` and `"merge"` class mappings.

5.3.4 Class creation mappings

`mapClass` (phase 5): One source class → one target class. Locates the source class by name in `inModel`, creates the target class in the configured target package, copies the source class's first `ownedComment` body as the definition, then iterates `getClassAttributeNames()` and calls `copyAttribute` for each.

`mergeClass` (phase 6): Two source classes → one target class. Requires exactly two source class names in the config. Each attribute entry names which source class it comes from via `fromSourceClass`. The class definition comes from the source class named in `definitionFrom`.

`newClass` (phase 7): No source class → one target class. Definition text comes from the config directly. No `Dependency` is created for `"new"` mappings.

5.3.5 Attribute copy mapping

`copyAttribute` handles multiplicity and definition overrides:

1. Looks up the source attribute via `requireAttributeByName`, which uses `getAllAttributes()` to include inherited attributes. This is necessary because the target class hierarchy is flat — source attributes defined on superclasses must still be reachable.
2. Creates the target `Property` with `name`, `isReadOnly`, and `type` copied from the source.
3. Applies multiplicity: if the config provides a lower or upper override for this attribute, it is used; otherwise the source property's bounds are copied.
4. Applies the definition: if the config provides a definition override, it is used; otherwise the source property's first `ownedComment` body is used.

Note: at phase 5–7, attribute `type` references still point into the source model. The type references are updated to target model copies in phase 8–9 (`copyDataTypes`).

5.3.6 DataType copy mappings

`copyDataTypes` (phase coordinator): After all classes and attributes exist, this mapping scans every `ownedAttribute` of every target class. For each attribute whose type is a `DataType` (but not a `PrimitiveType`), it calls `copyOneDataType` and then updates the attribute's type reference to the newly created target `DataType`.

`copyOneDataType` (recursive, tracking-guarded): Copies a single named DataType from the source model into the target model.

1. Guard: if `copiedTypeNames` already contains the name, returns immediately (prevents duplication and infinite recursion).
2. Marks the name as in-progress by adding it to `copiedTypeNames`.
3. Locates the source DataType via `findSourceDataTypeByName`.
4. Computes the source package path via `getDataTypePackagePath`, which walks up the namespace chain from the DataType's owner to the source model's main package, collecting package names. This path is reproduced under the target `dataTypesPackage`.
5. Navigates or creates each level of the sub-package path in the target model.
6. Creates the target `DataType`, copies its definition comment.
7. **Attribute flattening:** iterates `getAllDataTypeAttributesIncludingInherited()` — a recursive query that walks the generalization hierarchy and collects all attributes including inherited ones. The target DataType carries no generalization relationships; inherited attributes are placed directly on it. This ensures that a DataType like `Selector`, which inherits attributes from `ControlConstruct`, is fully usable in the target model without needing a class hierarchy.
8. For each attribute whose type is itself a structured DataType, calls `copyOneDataType` recursively and updates the type reference.
9. Draws in the full subtype tree via `copySubDataType`.

`copySubDataType` (recursive, no attribute flattening): Called from `copyOneDataType` and recursively from itself to draw in all concrete subtypes of a DataType that enters the target model.

Unlike `copyOneDataType`, it copies only `ownedAttribute` (no inherited attributes), because subtypes are concrete and carry their own definitions. It also:

- Takes a `superTypeName` parameter, looked up in the target model via `findTargetDataTypeByName`, and creates a `UML::Generalization` element on the target subtype pointing to the already-placed target supertype. This preserves the specialization relationship in the output model.
- Recurses into the subtype tree by calling `getDirectSubDataTypes`, passing `typeName` as the `superTypeName` for the next level.

The `copiedTypeNames` guard is shared between `copyOneDataType` and `copySubDataType`, so whichever path visits a type first wins, and the other path skips it without duplication.

5.3.7 Association creation mapping

`createAssociation` creates a UML Association with two `Property` member ends:

Navigability convention (UCMIS):

- The **object end** (the end pointing from the association toward the object class) is navigable: it is owned by the **subject class** via `ownedAttribute`, making it navigable from subject instances to object instances.
- The **subject end** is non-navigable: it is owned by the **association** via `ownedEnd`.
- Neither end has a role name (UCMIS convention).
- The association itself is owned by the **object class's package**.

Multiplicity: config overrides are checked first. If absent, the corresponding end of the source association is used. If the source association cannot be found, safe defaults are applied (`0..*` for the object end, `0..1` for the subject end).

Definition: config override takes precedence over the source association's first comment body.

5.3.8 Dependency creation mappings

`createDependencies` creates provenance Dependency elements:

- One Dependency per `"map"` source class (client = target class, supplier = source class proxy).
- Two Dependencies per `"merge"` target class (one for each source class).
- No Dependencies for `"new"` classes.

Each `Dependency`:

- Has `name = "TargetClass_dependsOn_SourceClass"`.
- Sets `client` to the target class element.
- Sets `supplier` to an unresolved EMF proxy created by `createSupplierProxy()`. The proxy carries the source class URI (`sourceModel.uri + "#" + sourceClass`) as its proxy URI. When the XMI is serialised, EMF writes this as `href="<uri>"` without requiring the source model to be loaded.
- Has an `ownedComment` whose body contains the full provenance text (see section 5.10).
- Is owned by the same package as the target (client) class.

5.3.9 Helper queries in `m2m.qvto` and `helpers.qvto`

Source model lookup:

- `findSourceClassByName` / `requireSourceClassByName` — uses `allSubobjectsOfKind(UML::Class)`.
- `findSourceAssociationByName` — uses `allSubobjectsOfKind(UML::Association)`.
- `findSourceDataTypeByName` — uses `allSubobjectsOfKind(UML::DataType)`.
- `getDataTypePackagePath` — walks `namespace` chain from the DataType's owner upward.

Target model lookup:

- `findTargetPackageByName` — scans `outModel.rootObjects()` for packages by name.
- `findTargetClassByName` — scans `outModel.rootObjects()` for classes by name.
- `findTargetDataTypeByName` — scans `outModel.rootObjects()` for DataTypes by name.

Multiplicity:

- `parseLower` / `parseUpper` — convert string bounds to integers (`"*"` → `-1`).
- `sourceLowerStr` / `sourceUpperStr` — extract string bounds from a source `Property`.
- `makeLower` / `makeUpper` — create `LiteralInteger` / `LiteralUnlimitedNatural` objects.

Comments:

- `makeComment(body)` — creates a `UML::Comment` with XML-character-escaped body (`<` → `<`, `>` → `>`).
- `firstCommentBody(element)` — returns the first `ownedComment` body or `""`.

DataType hierarchy:

- `getAllDataTypeAttributesIncludingInherited(dt)` — recursive; returns own attributes first, then appended parent attributes (no shadowing: if a name already exists, the parent attribute is skipped).
- `getDirectSubDataTypes(srcRoot, dt)` — returns all DataTypes in the source model that own a Generalization whose `general` is `dt`.

String utility:

- `toLowerFirst(s)` — converts the first character to lowercase for lowerCamelCase association end names.

5.3.10 `ConfigLib.qvto` — blackbox helper declarations

This library file declares all Java blackbox operations using the `blackbox helper` syntax. It contains no implementation — only signatures. QVTo 3.11 binds these to the Java methods in `JSONConfigLoader` via the extension point registration in `ucmism2m.blackbox/plugin.xml`.

Every method on `JSONConfigLoader` that is intended to be called from QVTo must have a corresponding declaration here. The method names and parameter types must match exactly.

5.4 Module: `ucmism2m.app`

Bundle symbolic name: `ucmism2m.app`
Purpose: Eclipse application entry point. Parses command-line arguments, drives the QVTo execution lifecycle, and serialises the output model.

5.4.1 UCMISTransformationApp

`src/ucmism2m/app/UCMISTransformationApp.java`

Implements `org.eclipse.equinox.app.IApplication`. Registered as the Eclipse application under the ID `ucmism2m.app.ucmism2m` in `plugin.xml`.

`start()` method:

1. Parses `-input`, `-output`, `-config` from `context.getArguments().get("application.args")`.
2. Validates all three arguments are present.
3. Calls `executeTransformation()`.

`executeTransformation()` method:

1. Initialises the UML2 package registry: `UMLPackage.eINSTANCE.eClass()`.
2. Creates a `ResourceSetImpl`, registers `.uml` extension with `UMLResource.Factory.INSTANCE`, and registers `UMLPackage.eINSTANCE` under its namespace URI.
3. Loads the input model resource from `inputPath`.
4. Creates `BasicModelExtent` for input (from loaded resource contents) and output (empty).
5. Creates a `TransformationExecutor` with URI `platform:/plugin/ucmism2m.transformation/transforms/m2m.qvto`.
6. Calls `executor.loadTransformation()` and checks for errors.
7. Creates `ExecutionContextImpl`, attaches a `WriterLog` to stdout, and sets the `configPath` configuration property.
8. Calls `executor.execute(executionContext, inputExtent, outputExtent)`.
9. Calls `saveOutputModel()`.

`saveOutputModel()` method:

1. Creates the output resource using `UMLResource.Factory.INSTANCE.createResource(outputURI)`. This produces a native `.uml` resource with a bare `<uml:Model>` root element and the correct XMI 2.5.1 namespace (`xmlns:xmi="http://www.omg.org/spec/XMI/20131001"`). No `<xmi:XMI>` wrapper is emitted.
2. Adds all output extent objects to the resource.
3. Calls `EcoreUtil.resolveAll(resourceSet)`.
4. Assigns a random UUID as `xmi:id` to every EObject that has none (`EcoreUtil.generateUUID()`). QVTo-created objects carry no IDs, so all output elements receive one on every run.
5. Saves with options: `OPTION_USE_XMI_TYPE` (writes `xmi:type` instead of `xsi:type`, eliminating the `xmlns:xsi` declaration), `OPTION_SAVE_ONLY_IF_CHANGED` (skips write if content is unchanged), and `OPTION_LINE_DELIMITER_UNSPECIFIED` (platform line endings).

5.5 Module: `ucmism2m.product`

Purpose: Eclipse product definition and platform assembly.

`ucmism2m.product` defines the product using `ucmism2m.product` (a `.product` file), which specifies the application ID, launcher name, JVM arguments, and the complete list of OSGi plugins required at runtime.

The product pom.xml uses `tycho-p2-director-plugin` with two executions: `materialize-products` (assembles the product) and `archive-products` (creates ZIP archives).

JVM arguments configured in the product file:

```
-Xms256m -Xmx1024m -Dosgi.requiredJavaVersion=21
```

The `-Dosgi.clean=true` flag can be appended here to force OSGi cache clearing on every launch (useful during development; not needed in production).

Platform profiles are defined in the parent `pom.xml`:

- `linux-only` (default) — builds for Linux x86_64 only.
- `all-platforms` — builds for Linux x86_64, Windows x86_64, macOS x86_64, macOS aarch64.
- `windows-only`, `macos-only` — single-platform variants.

5.6 Module: `ucmism2m.target`

File: `ucmism2m.target.target`
Purpose: Defines the Eclipse target platform — the set of Eclipse plugins available at compile time and used during product assembly.

Two p2 repository locations are declared:

Eclipse 2025-12 repository (`https://download.eclipse.org/releases/2025-12/`): Provides the Eclipse Platform, EMF Ecore/Common, UML2 5.x, Equinox executable, and Jackson 2.20.1 OSGi bundles (jackson-core, jackson-annotations, jackson-databind). Jackson is sourced from this repository (rather than Maven Central) because the Eclipse p2 repository provides proper OSGi manifests, keeping IDE (PDE) and CLI (Tycho) dependency resolution consistent.

QVTo 3.11.1 repository (`https://download.eclipse.org/mmt/qvto/builds/release/3.11.1/`): Provides the QVTo SDK and runtime feature.

Both locations use `includeAllPlatforms="true"` so that native fragments for all target platforms are resolved even when the build runs on a single OS.

5.7 The Mapping Configuration JSON

The JSON mapping configuration is the only file that users normally need to edit to produce different profile models. It drives the entire transformation.

5.7.1 Top-level structure

```json
{
  "$schema": "...",
  "transformation": { ... },
  "package": [ ... ],
  "mapping": {
    "class": [ ... ],
    "association": [ ... ]
  }
}
```

5.7.2 `transformation` object

```json
"transformation": {
  "description": "Human-readable description",
  "version": "1.0",
  "sourceModel": {
    "name": "DDI-CDI",
    "mainPackage": "DDI-CDI",
    "uri": "http://ddialliance.org/Specification/DDI-CDI/1.0/XMI/"
  },
  "targetModel": {
    "acronym": "DDSC",
    "modelTitle": "DDI Survey Components",
    "subTitle": "...",
    "majorVersion": "1",
    "minorVersion": "0",
    "language": "en",
    "uri": "http://example.org/DDSC/XMI/",
    "definition": "Full definition of the target model...",
    "mainPackage": "DDSC",
    "mainPackageDefinition": "...",
    "dataTypesPackage": "DDSC"
  }
}
```

`dataTypesPackage` controls where copied DataType sub-packages land. If it equals `mainPackage`, no separate data types package is created.

5.7.3 `package` array

```json
"package": [
  {
    "name": "Classes",
    "parent": "DDSC",
    "definition": "Contains all mapped classes and associations."
  }
]
```

Packages are created in config array order. A package may name another config-defined package as its parent (config order therefore matters for multi-level hierarchies).

5.7.4 `mapping.class` array — mapping types

`"map"` — one source class to one target class:

```json
{
  "mappingType": "map",
  "sourceClass": "InstanceVariable",
  "targetClass": "InstanceVariable",
  "targetPackage": "Classes",
  "attribute": [
    { "name": "name" },
    {
      "name": "displayLabel",
      "multiplicity": { "lower": "0", "upper": "*" },
      "definition": "Override definition text."
    }
  ],
  "sssom": [
    {
      "subject_label": "InstanceVariable",
      "object_label": "InstanceVariable",
      "predicate_id": "skos:exactMatch",
      "predicate_label": "exact match",
      "confidence": 1.0,
      "comment": ""
    }
  ]
}
```

`"merge"` — two source classes to one target class:

```json
{
  "mappingType": "merge",
  "sourceClass": ["DataStore", "WideDataSet"],
  "targetClass": "DataSet",
  "targetPackage": "Classes",
  "definitionFrom": "DataStore",
  "attribute": [
    { "name": "name", "fromSourceClass": "DataStore" },
    { "name": "purpose", "fromSourceClass": "WideDataSet" }
  ]
}
```

`"new"` — no source class:

```json
{
  "mappingType": "new",
  "targetClass": "CustomClass",
  "targetPackage": "Classes",
  "definition": "A new class with no DDI-CDI equivalent."
}
```

5.7.5 `mapping.association` array

```json
{
  "sourceAssociationName": "DataStore_has_LogicalRecord",
  "targetAssociationName": "DataStore_has_LogicalRecord",
  "objectClassMultiplicity": { "lower": "0", "upper": "*" }
}
```

`sourceAssociationName` and `targetAssociationName` must both follow the `SubjectClass_predicate_ObjectClass` convention. The subject and object class names are parsed from the target triple name; they must match exactly the target class names produced in phases 5–7. If `targetAssociationName` differs from `sourceAssociationName`, the target association gets its own name (fan-out from one source association to multiple target associations is supported — simply list multiple entries with the same `sourceAssociationName`).

5.7.6 Override priority rules

For any field where both a config override and a source model value are available, the config override always takes precedence. If the config override is absent (empty string or null), the source model value is used. There is no way to explicitly request "use source" — simply omit the override field.

5.8 Transformation Execution Flow

The following diagram shows the call sequence at runtime:

```
UCMISTransformationApp.start()
  └── executeTransformation()
      ├── Load UML ResourceSet, register UML2 factories
      ├── Load input .uml model → BasicModelExtent (inputExtent)
      ├── TransformationExecutor.loadTransformation()
      │    └── Loads m2m.qvto via platform:/plugin/... URI
      ├── ExecutionContextImpl.setConfigProperty("configPath", ...)
      └── TransformationExecutor.execute(ctx, inputExtent, outputExtent)
          └── m2m.main()
              ├── Phase 1: createTargetModel
              ├── Phase 2: createMainPackage
              ├── Phase 3: createMetadataPackage
              ├── Phase 4: createDefinedPackages (inlined)
              ├── Phase 5: mapClass (× N)
              │    └── copyAttribute (× M per class)
              ├── Phase 6: mergeClass (× N)
              │    └── copyAttribute (× M per class)
              ├── Phase 7: newClass (× N)
              ├── Phase 8-9: copyDataTypes
              │    └── copyOneDataType (recursive)
              │         └── copySubDataType (recursive)
              ├── Phase 10: createAssociation (× N)
              └── Phase 11: createDependencies
                   └── createOneDependency (× N)
      └── saveOutputModel()
          ├── Create UMLResource.Factory output resource
          ├── EcoreUtil.resolveAll()
          ├── Assign xmi:id UUIDs to all EObjects
          └── resource.save(options)
```

Each `configPath`-parameterised blackbox call in QVTo translates to a method call on `JSONConfigLoader`. The config is read from disk exactly once (first call) and served from cache thereafter.

5.9 Output Model Structure

The output `.uml` file is a native Eclipse UML2 model. Its root element is:

```xml
<uml:Model xmi:version="20131001"
        xmlns:xmi="http://www.omg.org/spec/XMI/20131001"
        xmlns:uml="http://www.eclipse.org/uml2/5.0.0/UML"
        xmi:id="...uuid..."
        name="DDSC">
```

The package hierarchy mirrors the config structure:

```
uml:Model (name = acronym, e.g. "DDSC")
├── ownedComment: model definition
├── packagedElement: main package (e.g. "DDSC")
│   ├── ownedComment: main package definition
│   ├── packagedElement: metadata package
│   │   └── packagedElement: metadata DataType
│   │       └── ownedAttribute × 7 (read-only String attributes)
│   ├── packagedElement: Classes package
│   │   ├── packagedElement: Class × N
│   │   │   ├── ownedComment: class definition
│   │   │   └── ownedAttribute × M (copied attributes)
│   │   │       └── ownedComment: attribute definition
│   │   ├── packagedElement: Association × N
│   │   │   ├── ownedComment: association definition
│   │   │   ├── memberEnd → (object end, owned by subject class)
│   │   │   └── ownedEnd → (subject end)
│   │   └── packagedElement: Dependency × N
│   │       ├── ownedComment: RST + RDF Turtle provenance
│   │       ├── client → target Class
│   │       └── supplier → proxy href to source class URI
│   └── packagedElement: DataType sub-packages × N (mirrored from source)
│       └── packagedElement: DataType × N
│           ├── ownedComment: DataType definition
│           ├── ownedAttribute × M (flattened, for supertypes)
│           └── generalization → supertype DataType (for subtypes)
```

5.10 Provenance and SSSOM

Each Dependency in the output model carries a UML Comment whose body contains two sections separated by a blank line.

Section 1 — reStructuredText field list:

Provenance
==========

Transformation Information

- Mapping Type: map
- Source Class: InstanceVariable
- Target Class: InstanceVariable

SSSOM Mapping

- Subject ID: http://ddialliance.org/Specification/DDI-CDI/1.0/XMI/#InstanceVariable
- Subject Label: InstanceVariable
- Object ID: http://example.org/DDSC/XMI/#InstanceVariable
- Object Label: InstanceVariable
- Predicate ID: skos:exactMatch
- Predicate Label: exact match
- Subject Category: UML Class
- Object Category: UML Class
- Confidence: 1.0
- Comment:

Section 2 — RDF Turtle:

```turtle
@prefix mapping: <http://example.org/transformation/mapping#> .
@prefix sssom:   <https://w3id.org/sssom#> .
@prefix src: <http://ddialliance.org/Specification/DDI-CDI/1.0/XMI/> .
@prefix tgt: <http://example.org/DDSC/XMI/> .

mapping:Mapping_InstanceVariable_InstanceVariable a sssom:Mapping ;
   sssom:subject_id <http://.../#InstanceVariable> ;
   sssom:subject_label "InstanceVariable" ;
   sssom:object_id <http://.../#InstanceVariable> ;
   sssom:object_label "InstanceVariable" ;
   sssom:predicate_id skos:exactMatch ;
   sssom:predicate_label "exact match" ;
   sssom:subjectCategory "UML Class" ;
   sssom:objectCategory "UML Class" ;
   sssom:confidence 1.0 ;
   sssom:comment "" ;
   mapping:mappingType "map" .
```

SSSOM fields (`subject_label`, `object_label`, `predicate_id`, `predicate_label`, `confidence`, `comment`) are taken from the `sssom` array in the class mapping config if present. If absent, defaults are used (`sourceClass` / `targetClass` as labels, `"sssom:mappingRelation"` as predicate, empty confidence and comment).

The Dependency supplier is an EMF proxy: it is not loaded into memory as a live object, but carries a proxy URI that EMF serialises as `href="..."` in the XMI output. This provides a machine-readable link from every target class back to its DDI-CDI source class without requiring the source model to be present at serialisation time.

5.11 Build System

5.11.1 Parent POM

`pom.xml` at the project root coordinates the build with:

- `groupId`: `org.ucmis`
- `artifactId`: `ucmism2m-parent`
- `version`: `1.0.0-SNAPSHOT`
- `tycho.version`: `5.0.0`
- Java source/target/release: `21`

Module build order is determined by Tycho from OSGi dependencies, not by the `<modules>` list order. Tycho automatically determines that `ucmism2m.target` must build first, then `ucmism2m.blackbox`, then `ucmism2m.transformation`, then `ucmism2m.app` and `ucmism2m.feature`, then `ucmism2m.product`.

5.11.2 Tycho lifecycle

Each module uses the Tycho packaging type:

Module	Packaging
`ucmism2m.target`	`eclipse-target-definition`
`ucmism2m.blackbox`	`eclipse-plugin`
`ucmism2m.transformation`	`eclipse-plugin`
`ucmism2m.app`	`eclipse-plugin`
`ucmism2m.feature`	`eclipse-feature`
`ucmism2m.product`	`eclipse-repository`

Tycho reads `MANIFEST.MF` (for `Require-Bundle` and `Export-Package`) and `plugin.xml` (for extension contributions) from each module to resolve OSGi dependencies. These files must be kept in sync with Java `import` statements and QVTo `import` directives.

5.11.3 OSGi cache management

The QVTo runtime extracts bundle JARs into `configuration/org.eclipse.osgi/` and caches them. Changes to QVTo scripts after a rebuild will not be reflected until the cache is invalidated. The `maven-clean-plugin` configuration in `ucmism2m.product/pom.xml` deletes the relevant cache directories as part of `mvn clean verify`, ensuring the latest QVTo scripts are always used in CI and release builds.

5.12 QVTo Constraints and Design Rules

Several hard constraints of QVTo 3.11 shaped the design of the transformation scripts. These are documented here as a reference for future maintainers.

Constraint	Description
`self` is input-only	In contextual mappings, `self` and all input parameters are read-only. Assignments must target `result` or local `var` variables.
`var` is only for local scope	Module-level mutable state must use `property` with `=` (not `:=`).
No `allInstances` on types	Use `element.allSubobjectsOfKind(OclType)` instead of the deprecated `allInstances()`.
No `first()` on Sets	Convert to Sequence first: `->asSequence()->first()`.
No `return` in mappings	Use nested `if/else/endif` blocks.
Single-statement `if/then/endif`	Must not have a trailing semicolon after the `then` branch. Multi-statement blocks require braces `{ }`.
`inModel` must not be shadowed	The global transformation input extent is named `inModel`. Local parameters and variables must not use this name.
Collections are value types	`OrderedSet` and `Sequence` have no mutating methods. Use `->including(x)` and reassign: `col := col->including(x)`.
Contextual mappings fail	Do not use contextual mappings (receiver `self`) for model element creation when `self` would be the input. Use plain (non-contextual) mappings and assign containment manually.
No `notNil()` / `isNil()`	Use `= null` and `<> null`.
No `Dict` type	Unsupported in QVTo 3.11. Use module-level `property` with `OrderedSet` or design around it.
No `configuration property` keyword	Use `property` at module level. Configuration properties are accessed via `configPath` passed as an argument through every helper.
Import paths are bundle-relative	`import ConfigLib;` resolves to `platform:/plugin/ucmism2m.transformation/transforms/ConfigLib.qvto`. Relative file paths do not work.
Blackbox naming	Avoid blackbox helper names that could conflict with OCL built-ins or QVTo keywords. Use explicit module qualification if ambiguity arises.
No `=` default on `property` with complex type	Use `OrderedSet{}` as the default value literal for `OrderedSet` properties.

End of UCMIS M2M Transformation Documentation